42 PROJECT ROADMAP / MANDATORY ONLY

FT_LS

Construye tu propio ls desde cero, entiende cada decisión y llega a la evaluación sabiendo defenderlo.

```
$ ./ft_ls -laRt
```

READY TO BUILD

7 fases • 42 tareas • pseudocódigo • checklist final

Preparada para Manuel Fernández

Tu misión

Avanza una etapa, comprueba el resultado y marca el progreso. Todo el “código” es pseudocódigo para que la solución final sea tuya, no de una IA.

Manuel, vamos a hacerlo desde cero

Vamos a construir ft_ls como si ahora mismo no tuvieras nada. No quiero que copies las casi 800 líneas del código generado por IA ni que intentes entenderlas todas de golpe. Ese camino solo sirve para acabar mirando la pantalla con cara de “esto funciona, pero no sé por qué”.

Aquí vamos a ir por capas: primero un ls tontísimo que abre un directorio y enseña nombres. Después aprenderás a guardar entradas, ordenar, interpretar argumentos y, solo cuando la base esté firme, añadirás -a, -r, -t, -l y -R.

No te voy a dar C real. Todo lo que parezca código será pseudocódigo. Tú tendrás que traducir cada idea a C, probarla y ser capaz de explicar por qué la has hecho así. De esa manera, cuando llegue la evaluación, el proyecto será tuyo y no de una IA.

La idea de esta guía

No intentes construir ft_ls entero hoy. Haz una tarea, comprueba que funciona y no avances hasta entenderla. Aquí no gana quien escribe más líneas, sino quien controla cada decisión.

El camino que vamos a seguir

01. Entender qué hace realmente ls
02. Construir el ls más tonto posible
03. Guardar entradas y controlar la memoria
04. Ordenar por nombre y por tiempo
05. Interpretar argumentos y flags
06. Construir -l pieza a pieza
07. Procesar archivos y directorios
08. Añadir -R sin crear un monstruo
09. Revisar errores, fugas y Makefile
10. Superar la checklist final

Nuestra regla

Primero leer. Después guardar. Después ordenar. Al final imprimir. Cada flag toca una fase distinta. Si mezclas todo desde el principio, esto se convierte en una feria, y no precisamente de las buenas.

0. Antes de programar: ¿qué hace realmente ls?

Cuando escribes ls, parece que el sistema simplemente dice: “toma, aquí tienes los archivos”. Pero por debajo hay bastante más movimiento. Tu programa tendrá que interpretar lo que pide el usuario, decidir qué rutas procesar, obtener información del sistema, ordenar y mostrar el resultado.

Tu mandatory pide exactamente -l, -R, -a, -r y -t, además de sus combinaciones. También exige errores razonables, cero fugas y un Makefile que genere ft_ls. No te hace falta meterte en columnas, ACL, atributos extendidos ni veinte opciones más. No te vengas arriba todavía, figura.

Qué hace ls internamente

```
RECIBIR argumentos
INTERPRETAR opciones
DETERMINAR rutas
PARA CADA ruta:
    COMPROBAR su tipo
    SI es archivo: obtener metadata y guardarlo
    SI es directorio: abrir, leer, filtrar y guardar entradas
ORDENAR entradas
MOSTRAR entradas
```

Responsabilidad de cada flag

Flag	Modifica	Idea
-a	Filtrado	Incluye nombres que comienzan por punto
-r	Ordenación	Invierte el orden final
-t	Ordenación	Usa la fecha de modificación
-l	Presentación	Muestra metadata detallada
-R	Recorrido	Entra recursivamente en subdirectorios

Diseño correcto

No programes casos como “si -t y -r” o “si -a y -l”. Cada flag cambia una propiedad independiente: filtro, comparador, formato o recorrido.

1. El mapa completo, para que sepas adónde vamos

```
MAIN
  -> inicializar flags
  -> parsear argumentos
  -> obtener rutas
  -> clasificar archivos y directorios
  -> procesar archivos
  -> procesar directorios
       -> leer entradas
       -> filtrar (-a)
       -> obtener metadata
       -> ordenar (-t / -r)
       -> mostrar (normal / -l)
       -> recorrer subdirectorios (-R)
```

Quédate con este mapa. No tienes que implementarlo entero ahora. Solo quiero que entiendas que cada pieza tiene su trabajo. La función que lee directorios no pinta permisos, y la que formatea fechas no abre rutas. Si cada función hace de todo, luego llega -R y te pega una guantá

de realidad.

Estructuras conceptuales

```
ESTRUCTURA Flags
    long_format
    recursive
    show_all
    reverse
    sort_time
FIN

ESTRUCTURA Entrada
    nombre
    ruta_completa
    metadata
FIN

ESTRUCTURA Lista
    entradas
    cantidad
    capacidad
FIN
```

FASE 1. Vamos a crear el ft_ls más tonto posible

Primera misión: conseguir que ./ft_ls abra el directorio actual, lea entradas, muestre nombres y cierre. Nada de flags, fechas ni recursividad. No corras, que todavía estamos poniendo los cimientos.

3.1 Abrir, leer y cerrar un directorio

opendir abre un directorio; readdir entrega una entrada cada vez; closedir libera el recurso asociado al directorio.

```
FUNCION listar_directorio_minimo(ruta)
    directorio = ABRIR_DIRECTORIO(ruta)
    SI directorio no pudo abrirse
        MOSTRAR error
        DEVOLVER fracaso
    FIN

    MIENTRAS exista otra entrada
        entrada = LEER_SIGUIENTE_ENTRADA(directorio)
        MOSTRAR entrada.nombre
    FIN

    CERRAR_DIRECTORIO(directorio)
    DEVOLVER exito
FIN
```

3.2 Ocultar archivos que empiezan por punto

```
FUNCION es_visible_sin_a(nombre)
    SI primer_caracter(nombre) ES punto
        DEVOLVER falso
    FIN
    DEVOLVER verdadero
FIN
```

Al principio, usa esa función antes de imprimir. Más tarde será reemplazada por una función que también reciba las flags.

3.3 El cambio arquitectónico esencial

```
INCORRECTO PARA EL PROYECTO FINAL
    leer -> imprimir -> leer -> imprimir

CORRECTO
    leer todo -> guardar todo -> ordenar -> imprimir
```

Criterio de finalización

No avances hasta que puedas abrir “.”, recorrer sus entradas y cerrar el directorio sin errores. Después oculta correctamente los nombres que empiezan por punto.

FASE 2. Ya no basta con leer: ahora toca guardar

4.1 Por qué guardas nombre y ruta

Para mostrar una entrada puede bastar con main.c, pero para pedir información al sistema necesitas una ruta como src/main.c. Por eso conviene conservar ambos valores.

4.2 Construir una ruta

```
FUNCION unir_ruta(directorio, nombre)
    calcular longitud necesaria
    reservar una nueva cadena
    copiar directorio
    SI directorio no termina en separador
        añadir separador
    FIN
    añadir nombre
    DEVOLVER nueva cadena
FIN
```

4.3 stat frente a lstat

stat sigue un enlace simbólico y describe su destino. lstat describe el enlace en sí. En ft_ls, lstat es clave para imprimir correctamente symlinks y evitar tratarlos accidentalmente como directorios reales durante -R.

```
FUNCION crear_entrada(ruta, nombre)
    metadata = CONSULTAR_CON_LSTAT(ruta)
    SI la consulta falla
        MOSTRAR error asociado a ruta
        DEVOLVER fracaso
    FIN
    nueva.nombre = COPIAR(nombre)
    nueva.ruta_completa = COPIAR(ruta)
    nueva.metadata = metadata
    DEVOLVER nueva
FIN
```

Ahora imagina que el directorio tiene 200 archivos. No puedes inventarte 200 variables. Necesitas una caja que crezca cuando se llena. Esa caja será tu lista dinámica.

4.4 Lista dinámica

La lista tiene una cantidad de elementos usados y una capacidad reservada. Cuando la cantidad alcanza la capacidad, se amplía. Un crecimiento típico duplica la capacidad para no reservar memoria en cada inserción.

```
FUNCION inicializar_lista(lista)
    lista.entradas = vacio
    lista.cantidad = 0
    lista.capacidad = 0
FIN

FUNCION añadir_entrada(lista, entrada)
    SI lista.cantidad ES IGUAL A lista.capacidad
        ampliar_lista(lista)
    FIN
    lista.entradas[lista.cantidad] = entrada
    lista.cantidad = lista.cantidad + 1
FIN

FUNCION ampliar_lista(lista)
    SI lista.capacidad ES 0
        nueva_capacidad = capacidad_inicial
    SINO
        nueva_capacidad = lista.capacidad * 2
    FIN
    ampliar memoria conservando contenido
    actualizar capacidad
FIN
```

4.5 Propiedad de memoria

```
FUNCION liberar_lista(lista)
    PARA cada entrada de lista
        liberar entrada.nombre
        liberar entrada.ruta_completa
    FIN
    liberar array de entradas
    reiniciar cantidad y capacidad
FIN
```

Pregunta obligatoria

Cada vez que reserves memoria, responde inmediatamente: “¿quién será responsable de liberarla?”. Si no conoces la respuesta, todavía no está bien diseñado.

4.6 Lectura definitiva de un directorio

```
FUNCION leer_directorio(ruta, flags, lista)
    directorio = ABRIR_DIRECTORIO(ruta)
    SI falla
        MOSTRAR error
        DEVOLVER fracaso
    FIN

    MIENTRAS exista otra entrada
        objeto = LEER_SIGUIENTE_ENTRADA(directorio)
        SI debe_mostrarse(objeto.nombre, flags)
            ruta_completa = unir_ruta(ruta, objeto.nombre)
            entrada = crear_entrada(ruta_completa, objeto.nombre)
            SI entrada fue creada
                añadir_entrada(lista, entrada)
            FIN
            liberar ruta_completa temporal
        FIN
    FIN

    CERRAR_DIRECTORIO(directorio)
    DEVOLVER exito
FIN
```

FASE 3. Poner orden en el corral

Hasta ahora readdir te entrega nombres en el orden que quiera el sistema. Ese orden no manda. Tú vas a leerlo todo, guardarlo y después decidir cómo ordenarlo.

5.1 Separar algoritmo y comparador

El algoritmo mueve los elementos. El comparador decide cuál debe ir antes. Esta separación permite reutilizar la misma ordenación para nombre, tiempo y orden inverso.

```
FUNCION comparar_por_nombre(A, B)
    SI nombre_A va antes que nombre_B
        DEVOLVER negativo
    SI nombre_A va despues que nombre_B
        DEVOLVER positivo
    DEVOLVER cero
FIN
```

5.2 Orden por tiempo

```
FUNCION comparar_por_tiempo(A, B)
    SI tiempo_A ES MAS RECIENTE que tiempo_B
        DEVOLVER negativo
    SI tiempo_A ES MAS ANTIGUO que tiempo_B
        DEVOLVER positivo
    DEVOLVER comparar_por_nombre(A, B)
FIN
```

Usar el nombre como desempate evita resultados arbitrarios cuando dos entradas tienen la misma fecha. Cuando llegues a las pruebas avanzadas, compara también la precisión subsegundo disponible en la metadata.

5.3 Comparador general con -t y -r

```
FUNCION comparar_entradas(A, B, flags)
    SI flags.sort_time
        resultado = comparar_por_tiempo(A, B)
    SINO
        resultado = comparar_por_nombre(A, B)
    FIN

    SI flags.reverse
        resultado = resultado * -1
    FIN
    DEVOLVER resultado
FIN
```

FASE 4. Entender qué le está pidiendo el usuario

Ahora sí entran las flags. Antes no. Si has llegado aquí con lectura, lista, ordenación e impresión separadas, vas bien. Si no, para y arregla la base antes de seguir.

6.1 Inicializar las flags

```
FUNCION inicializar_flags(flags)
    flags.long_format = falso
    flags.recursive = falso
    flags.show_all = falso
    flags.reverse = falso
    flags.sort_time = falso
FIN
```

6.2 Una opción puede contener varias letras

```
FUNCION interpretar_opcion(argumento, flags)
    empezar despues del primer guion
    PARA cada caracter restante
        SEGUN caracter
            l: activar long_format
            R: activar recursive
            a: activar show_all
            r: activar reverse
            t: activar sort_time
            otro: mostrar error y devolver fracaso
        FIN SEGUN
    FIN
    DEVOLVER exito
FIN
```

6.3 Separar opciones y rutas

```
FUNCION interpretar_argumentos(argumentos, flags, rutas)
    opciones_activas = verdadero
    PARA cada argumento despues del nombre del programa
        SI argumento ES exactamente doble_guion
            opciones_activas = falso
        SINO SI opciones_activas Y parece una opcion valida
            interpretar_opcion(argumento, flags)
        SINO
            guardar argumento como ruta
        FIN
    FIN
    SI no hay rutas
        guardar punto como ruta por defecto
    FIN
FIN
```

6.4 Implementar -a

```
FUNCION debe_mostrarse(nombre, flags)
    SI flags.show_all
        DEVOLVER verdadero
    SI nombre empieza por punto
        DEVOLVER falso
    DEVOLVER verdadero
FIN
```

6.5 Resultado de las combinaciones

- Sin flags: filtrar ocultos, ordenar por nombre, mostrar nombre.
- -a: no filtrar ocultos.
- -r: invertir el comparador final.
- -t: comparar por tiempo y desempatar por nombre.
- -tr o -rt: comparar por tiempo y después invertir.

FASE 5. El jefe final: construir -l sin volverte loco

Aquí empieza la parte gorda. -l parece una única opción, pero en realidad junta tipo, permisos, enlaces, usuario, grupo, tamaño, fecha, nombre y symlinks. Si intentas hacerlo en una sentada, te vas a liar. Lo vamos a cortar en trocitos manejables.

7.1 Tipo de entrada

```
FUNCION obtener_tipo(metadata)
    SI es directorio: DEVOLVER 'd'
    SI es enlace simbolico: DEVOLVER 'l'
    SI es dispositivo de caracteres: DEVOLVER 'c'
    SI es dispositivo de bloques: DEVOLVER 'b'
    SI es FIFO: DEVOLVER 'p'
    SI es socket: DEVOLVER 's'
    DEVOLVER '-'
FIN
```

7.2 Permisos normales

```
FUNCION construir_permisos(metadata)
    resultado[0] = obtener_tipo(metadata)
    resultado[1..3] = permisos de propietario: r, w, x o '-'
    resultado[4..6] = permisos de grupo: r, w, x o '-'
    resultado[7..9] = permisos de otros: r, w, x o '-'
    terminar la cadena correctamente
    DEVOLVER resultado
FIN
```

7.3 Permisos especiales

Después del rwx básico, añade setuid, setgid y sticky bit. Sustituyen posiciones de ejecución por s/S/t/T según exista o no permiso de ejecución normal.

```
SI setuid esta activo
    SI propietario puede ejecutar: posicion_propietario = 's'
    SINO: posicion_propietario = 'S'

SI setgid esta activo
    SI grupo puede ejecutar: posicion_grupo = 's'
    SINO: posicion_grupo = 'S'

SI sticky esta activo
    SI otros pueden ejecutar: posicion_otros = 't'
    SINO: posicion_otros = 'T'
```

7.4 Enlaces, propietario y grupo

El número de hard links ya llega en la metadata. El sistema también proporciona UID y GID; debes intentar convertirlos en nombres. Si no existe una entrada de usuario o grupo, utiliza el identificador numérico como respaldo.

```
FUNCION obtener_nombre_usuario(uid)
    usuario = BUSCAR_USUARIO(uid)
    SI existe: DEVOLVER usuario.nombre
    DEVOLVER uid convertido a texto
FIN

FUNCION obtener_nombre_grupo(gid)
    grupo = BUSCAR_GRUPO(gid)
    SI existe: DEVOLVER grupo.nombre
    DEVOLVER gid convertido a texto
FIN
```

7.5 Tamaño, dispositivos y fecha

Para archivos normales muestra el tamaño almacenado en la metadata. Para dispositivos de caracteres o bloques, investiga los números major y minor. Para la fecha, convierte el tiempo de modificación a una representación legible y, después, reproduce el cambio entre hora y año para archivos antiguos.

```
FUNCION formatear_fecha(tiempo_modificacion, tiempo_actual)
    convertir timestamp en fecha local
    SI el archivo se considera reciente
        formato = mes + dia + hora + minuto
    SINO
        formato = mes + dia + año
    FIN
    DEVOLVER formato
FIN
```

7.6 Enlaces simbólicos

```
FUNCION mostrar_destino_enlace(entrada)
    SI entrada no es enlace simbolico
        TERMINAR
    destino = LEER_DESTINO_DEL_ENLACE(entrada.ruta_completa)
    SI falla
        MOSTRAR error
        TERMINAR
    MOSTRAR flecha y destino
FIN
```

7.7 Imprimir una entrada larga

```
FUNCION mostrar_largo(entrada, anchos)
    permisos = construir_permisos(entrada.metadata)
    usuario = obtener_nombre_usuario(entrada.uid)
    grupo = obtener_nombre_grupo(entrada.gid)
    fecha = formatear_fecha(entrada.mtime)

    mostrar permisos
    mostrar numero de enlaces alineado
    mostrar usuario y grupo alineados
    mostrar tamaño o major/minor
    mostrar fecha
    mostrar nombre
    si es symlink: mostrar destino
    nueva linea
FIN
```

7.8 Calcular anchos antes de imprimir

Para alinear las columnas, recorre primero la lista y calcula la anchura máxima del número de enlaces, usuario, grupo y tamaño. Después imprime todas las entradas usando esos máximos.

```
FUNCION calcular_anchos(lista)
    anchos = valores minimos
    PARA cada entrada
        actualizar maximo de enlaces
        actualizar maximo de usuario
        actualizar maximo de grupo
        actualizar maximo de tamaño o dispositivo
    FIN
    DEVOLVER anchos
FIN
```

7.9 total

```
FUNCION calcular_total(lista)
    total_bloques = 0
    PARA cada entrada
        total_bloques += bloques_asignados_de_entrada
    FIN
    convertir a la unidad usada por la salida esperada
    DEVOLVER total
FIN
```

No implementes columnas

Cuando no uses -l, puedes imprimir una entrada por línea. La salida multicolumna pertenece al bonus y añade complejidad innecesaria.

FASE 6. Cuando el usuario te pasa de todo un poco

8.1 Clasificar cada ruta

```
FUNCION clasificar_rutas(rutas, archivos, directorios)
    PARA cada ruta
        metadata = CONSULTAR_CON_LSTAT(ruta)
        SI falla
            mostrar error
            registrar estado de fallo
            CONTINUAR
        FIN
        SI representa directorio real
            añadir a directorios
        SINO
            añadir a archivos
        FIN
    FIN
FIN
```

8.2 Orden correcto de procesamiento

```
ordenar archivos con las flags
ordenar directorios con las flags
mostrar archivos
PARA cada directorio
    si hay varios argumentos: mostrar cabecera con su ruta
    listar contenido del directorio
FIN
```

Mantén separado el nombre que se muestra del camino que se utiliza internamente. Si el usuario escribe una ruta explícita como src/main.c, no pierdas esa información.

FASE 7. -R: la función que se llama a sí misma

Deja -R para casi el final. La recursividad no arregla problemas: los multiplica. Si listar un único directorio funciona perfecto, entonces -R consiste en reutilizar esa misma función con cada subdirectorio.

9.1 La función se reutiliza a sí misma

```
FUNCION listar_directorio(ruta, flags, mostrar_cabecera)
    lista = lista vacia
    leer_directorio(ruta, flags, lista)
    ordenar_lista(lista, flags)
    mostrar contenido de lista

    SI flags.recursive
        PARA cada entrada ordenada
            SI entrada es directorio real
               Y nombre no es punto
               Y nombre no es punto_punto
                    mostrar separador y cabecera
                    listar_directorio(entrada.ruta_completa, flags, verdadero)
            FIN
        FIN
    FIN

    liberar_lista(lista)
FIN
```

9.2 Evitar recursividad infinita

```
FUNCION es_punto_o_punto_punto(nombre)
    SI nombre ES punto: DEVOLVER verdadero
    SI nombre ES punto_punto: DEVOLVER verdadero
    DEVOLVER falso
FIN
```

No sigas un symlink como directorio durante el recorrido. Para decidirlo, utiliza la metadata obtenida con lstat sobre la entrada en sí.

Orden importante

Primero muestra el contenido del directorio actual. Después recorre sus subdirectorios en el mismo orden ya calculado. Así -t y -r también afectan al recorrido recursivo.

FASE 8. Que falle con dignidad y no deje memoria tirada

10.1 Error fatal frente a recuperable

• Fallo al reservar memoria: normalmente fatal, porque no puedes mantener una estructura consistente.

• Ruta inexistente o directorio sin permiso: muestra el error, establece un estado de salida no exitoso y continúa con los demás argumentos.

• Fallo al leer un symlink concreto: informa del problema sin destruir estructuras dos veces.

10.2 Patrón general

```
resultado = REALIZAR_OPERACION
SI resultado indica error
    guardar el error antes de realizar otras operaciones
    mostrar un mensaje asociado al nombre correcto
    actualizar estado global de salida
    liberar solo los recursos que ya pertenecen a esta funcion
    decidir si continuar o terminar
FIN
```

10.3 Reglas de memoria

• Toda reserva tiene exactamente un propietario.

• Toda transferencia de propiedad debe estar clara.

• Liberar una lista incluye liberar el contenido de sus entradas.

• Después de transferir un puntero a una estructura, no lo liberes también desde la función anterior.

• Al fallar a mitad de una función, libera únicamente aquello que ya fue creado.

10.4 Makefile conceptual

```
VARIABLES
    nombre = ft_ls
    compilador
    flags de compilacion
    fuentes
    objetos

REGLAS
    all: construir ejecutable
    cada objeto: recompilar solo si cambia su fuente o cabeceras
    ft_ls: enlazar objetos
    clean: eliminar objetos
    fclean: clean + eliminar ejecutable
    re: reconstruccion completa
```

Tu camino real: 42 tareas, una detrás de otra

01 Crear el punto de entrada

Meta: El programa arranca y termina

• Sin flags ni lectura todavía.

02 Abrir el directorio actual

Meta: Dominar la apertura y el cierre

• Abrir “.” y gestionar el fallo.

• Cerrar siempre si la apertura tuvo éxito.

03 Leer entradas

Meta: Crear el primer mini-ls

• Recorrer hasta que no queden entradas.

• Mostrar temporalmente cada nombre.

04 Ocultar nombres con punto

Meta: Imitar el comportamiento básico

• Ignorar nombres cuyo primer carácter sea punto.

05 Diseñar Entrada

Meta: Definir la unidad de información

• Separar nombre, ruta y metadata.

06 Unir rutas

Meta: Construir caminos seguros

• Calcular tamaño, separador y terminador.

07 Usar lstat

Meta: Obtener metadata sin seguir symlinks

• Gestionar errores por entrada.

08 Crear Lista

Meta: Guardar un número variable de entradas

• Cantidad, capacidad y memoria.

09 Cambiar lectura por almacenamiento

Meta: Dejar de imprimir desde readdir

• Leer, crear entrada y añadir.

10 Comparar por nombre

Meta: Definir el orden base

• Comparador independiente.

11 Ordenar la lista

Meta: Aplicar el comparador

• Probar casos ya ordenados e inversos.

12 Separar impresión

Meta: Completar el flujo base

• Leer, ordenar, imprimir y liberar.

13 Crear Flags

Meta: Modelar opciones

• Inicializar todo a falso.

14 Parsear -l

Meta: Aprender el parser

• Activar una bandera sin aplicar comportamiento.

15 Añadir las cinco flags

Meta: Reconocer todas las opciones obligatorias

• Errores para caracteres desconocidos.

16 Combinar opciones

Meta: Aceptar -laRt y similares

• Recorrer las letras después del guion.

17 Implementar -a

Meta: Modificar el filtro

• Comprobar punto, punto-punto y ocultos.

18 Implementar -r

Meta: Invertir el comparador

• No invertir manualmente la lista.

19 Implementar -t

Meta: Ordenar por modificación

• Usar nombre como desempate.

20 Tipo de archivo

Meta: Primer carácter de -l

• Normal, directorio, link y especiales.

21 Permisos rwx

Meta: Crear nueve posiciones

• Propietario, grupo y otros.

22 Permisos especiales

Meta: Cubrir s/S/t/T

• Probar todas las combinaciones.

23 Número de enlaces

Meta: Mostrar metadata

• Alinear después, no ahora.

24 Usuario

Meta: Resolver UID

• Fallback numérico.

25 Grupo

Meta: Resolver GID

• Fallback numérico.

26 Tamaño

Meta: Mostrar bytes

• No leer el contenido del archivo.

27 Fecha

Meta: Formatear mtime

• Primero formato básico, después edge cases.

28 Destino de symlink

Meta: Usar lectura de enlace

• Mostrar flecha y destino.

29 Alineación

Meta: Calcular máximos

• Dos pasadas: medir e imprimir.

30 Total

Meta: Sumar bloques asignados

• Aplicar solo al contenido de directorios con -l.

31 Ruta por defecto

Meta: Usar punto sin argumentos

• No convertirla en un caso especial gigante.

32 Una ruta explícita

Meta: Procesar archivo o directorio

• Detectar el tipo.

33 Varios directorios

Meta: Iterar argumentos

• Cabeceras y separadores.

34 Mezcla de tipos

Meta: Separar archivos y directorios

• Mostrar archivos antes del contenido de directorios.

35 Rutas inexistentes

Meta: Continuar tras errores

• Estado final no exitoso.

36 Activar -R

Meta: Llamada recursiva

• Reutilizar listar_directorio.

37 Excluir . y ..

Meta: Evitar bucles

• Comparación exacta del nombre.

38 No seguir symlinks

Meta: Evitar recorrer destinos

• Decidir con lstat.

39 Combinar -R

Meta: Probar -Ra, -Rl, -Rt, -Rr

• El orden del recorrido debe respetar flags.

40 Auditar reservas

Meta: Eliminar fugas

• Mapa de propiedad de cada puntero.

41 Auditar operaciones del sistema

Meta: Cubrir fallos

• Apertura, lectura, metadata y symlinks.

42 Validar con Makefile y herramientas

Meta: Cerrar el mandatory

• Build limpio, segundo make sin recompilar, Valgrind y diff.

Ahora toca demostrar que funciona de verdad

12.1 Comparación diferencial

Para cada combinación, ejecuta el ls del sistema y ft_ls bajo el mismo entorno. Separa salida estándar y salida de error. Compara ambos resultados y estudia cada diferencia, en vez de asumir que tu salida “parece correcta”.

```
EJECUTAR ls del sistema con una combinacion
GUARDAR salida normal y errores
EJECUTAR ft_ls con la misma combinacion
GUARDAR salida normal y errores
COMPARAR ambos pares de archivos
ANALIZAR cada diferencia
```

12.2 Casos que debes crear

• Archivo normal, vacío y grande.

• Directorio vacío y con muchos elementos.

• Archivos y directorios ocultos.

• Symlink a archivo, a directorio y symlink roto.

• FIFO, socket y, si el entorno lo permite, dispositivos.

• Nombres con espacios y nombres largos.

• Permisos especiales setuid, setgid y sticky.

• Tiempos iguales, tiempos diferentes y precisión subsegundo.

• Directorio sin permiso de lectura.

• Varias rutas mezclando archivos, directorios y rutas inexistentes.

12.3 Checklist de entrega

☐ ./ft_ls sin argumentos

☐ archivo explícito

☐ directorio explícito

☐ varios argumentos

☐ -a

☐ -r

☐ -t

☐ -l

☐ -R

☐ -la

☐ -ltr

☐ -laRt

☐ opciones combinadas

☐ opción inválida

☐ ruta inexistente

☐ directorio inaccesible

☐ symlink normal

☐ symlink roto

☐ permisos especiales

☐ usuario y grupo

☐ tamaño o major/minor

☐ fecha antigua y reciente

☐ total

☐ sin recursividad en . y ..

☐ sin seguir symlink durante -R

☐ sin segfault

☐ sin double free

☐ sin fugas

☐ Makefile genera ft_ls

☐ segundo make no recompila

La ruta final, sin atajos ni humo

```
ft_ls minimo
  -> lectura
  -> estructura Entrada
  -> lista dinamica
  -> ordenacion
  -> parser
  -> -a
  -> -r
  -> -t
  -> -l
  -> multiples argumentos
  -> -R
  -> errores
  -> memoria
  -> Makefile
  -> comparacion con ls
```

La meta de verdad

No quiero que reproduzcas cientos de líneas de una IA. Quiero que puedas decir: “leo con readdir por esto, guardo antes de ordenar por esto, esta cadena la libera esta estructura, uso lstat por los symlinks y -R reutiliza mi función de directorios”. Cuando puedas explicar eso sin mirar, el proyecto ya es tuyo, miarma.

Cuando termines cada fase, apunta con tus palabras qué recibe cada función, qué devuelve, qué memoria crea, quién la libera, qué puede fallar y cómo lo has probado. No es burocracia: es lo que hará que puedas defender el proyecto sin depender del código de la IA.

Empieza solo por las tareas 1 a 4. Cuando puedas abrir ., recorrerlo y mostrar únicamente entradas visibles entendiendo cada operación, entonces pasas a Entrada y a la lista. Paso corto, prueba clara y pa’lante. Así se levanta un ft_ls serio, miarma.